

UNIVERSIDAD DA VINCI DE GUATEMALA



Tema:

Evaluación Final

Estudiante:

Sharloth Yessenia Betzabé Alvarado Morales

Carné: 202504130

Curso: Estructuras de Datos

Catedrático Tutor: Ingeniero Brandon Chitay

Mayo 2026

índice

I.	FASE 1: Fichas Técnicas	3
1.1.	Ficha Técnica 1: Árbol Binario de Búsqueda (BST)	3
1.2.	Ficha Técnica 2: Árbol Balanceado (AVL)	4
1.3.	Ficha Técnica 3: Árbol B / B+.....	5
1.4.	Ficha Técnica 4: Trie (Árbol de Prefijos)	6
1.5.	Ficha Técnica 5: Heap (Montículo Min/Max)	7

I. FASE 1: Fichas Técnicas

1.1.Ficha Técnica 1: Árbol Binario de Búsqueda (BST)

identificación: Estructura de datos arbórea no lineal donde cada nodo tiene como máximo dos hijos. El subárbol izquierdo contiene valores menores al nodo raíz y el subárbol derecho valores mayores.

Análisis de Complejidad (Big O):

Búsqueda: Caso Promedio: $O(\log n)$ | Peor Caso: $O(n)$ (cuando el árbol se desbalancea y se convierte en una lista enlazada).

Inserción: Caso Promedio: $O(\log n)$ | Peor Caso: $O(n)$.

Eliminación: Caso Promedio: $O(\log n)$ | Peor Caso: $O(n)$.

Características Mecánicas: Organiza los elementos de forma jerárquica mediante comparaciones directas en cada inserción. No cuenta con mecanismos automáticos de reestructuración, por lo que su forma final depende estrictamente del orden en que se insertan los datos.

Ventajas y Limitaciones:

Mayor Ventaja: Muy simple de implementar y altamente eficiente si los datos se insertan de forma aleatoria.

Mayor Limitación: Carece de balanceo; si los datos entran ordenados, pierde su eficiencia y la complejidad se degrada a lineal $O(n)$.

Caso de Uso en la Industria: Utilizado en sistemas de enrutamiento de redes simples y para construir árboles de expresiones algebraicas en compiladores.

1.2.Ficha Técnica 2: Árbol Balanceado (AVL)

Identificación: Es un Árbol Binario de Búsqueda auto-balanceado. Recibe su nombre por sus creadores Adelson-Velskii y Landis.

Análisis de Complejidad (Big O):

Búsqueda: Caso Promedio: $O(\log n)$ | Peor Caso: $O(\log n)$.

Inserción: Caso Promedio: $O(\log n)$ | Peor Caso: $O(\log n)$.

Eliminación: Caso Promedio: $O(\log n)$ | Peor Caso: $O(\log n)$.

Características Mecánicas: Mantiene el balance mediante el Factor de Balance (FB) de cada nodo ($FB = \text{altura derecho} - \text{altura izquierdo}$). El FB de cualquier nodo solo puede ser -1, 0 o 1. Si tras una inserción o eliminación el FB es menor a -1 o mayor a 1, se aplican rotaciones (simples o dobles) para reequilibrarlo.

Ventajas y Limitaciones:

Mayor Ventaja: Garantiza operaciones de búsqueda extremadamente rápidas en el peor escenario posible debido a su estricto balanceo.

Mayor Limitación: Las operaciones de inserción y eliminación son costosas en tiempo de procesamiento por la necesidad de calcular alturas y realizar rotaciones frecuentes.

Caso de Uso en la Industria: Bases de datos de memoria integrada e indexación de diccionarios que requieren búsquedas masivas y constantes pero pocas modificaciones (lectura intensiva).

1.3.Ficha Técnica 3: Árbol B / B+

Identificación: Árboles de búsqueda auto-balanceados balanceados por altura, optimizados para sistemas de almacenamiento secundario (discos duros/SSD). A diferencia de los binarios, sus nodos pueden tener cientos de hijos.

Análisis de Complejidad (Big O):

Búsqueda: Caso Promedio: $O(\log n)$ | Peor Caso: $O(\log n)$.

Inserción: Caso Promedio: $O(\log n)$ | Peor Caso: $O(\log n)$.

Eliminación: Caso Promedio: $O(\log n)$ | Peor Caso: $O(\log n)$.

Características Mecánicas: Los nodos manejan múltiples claves y punteros a hijos. El árbol crece hacia arriba cuando los nodos se llenan y se dividen (split). En el Árbol B+, todas las claves y datos se duplican/guardan estrictamente en las hojas, y estas hojas se conectan secuencialmente mediante una lista enlazada.

Ventajas y Limitaciones:

Mayor Ventaja: Minimiza los accesos a disco (I/O) al empaquetar bloques de datos masivos en un solo nodo. El Árbol B+ permite rangos de búsqueda secuenciales ultra rápidos.

Mayor Limitación: Gran complejidad de código y consumo extra de memoria por la duplicidad de claves en las hojas (en el caso de B+).

Caso de Uso en la Industria: Motores de bases de datos relacionales como PostgreSQL, MySQL (InnoDB), y sistemas de archivos modernos como NTFS y APFS.

1.4.Ficha Técnica 4: Trie (Árbol de Prefijos)

Identificación: Árbol de búsqueda ordenado que se utiliza para almacenar cadenas de texto de manera asociativa, aprovechando los prefijos comunes entre palabras.

Análisis de Complejidad (Big O): (Donde k es la longitud de la palabra/llave a buscar)

Búsqueda: Caso Promedio: $O(k)$ | Peor Caso: $O(k)$.

Inserción: Caso Promedio: $O(k)$ | Peor Caso: $O(k)$.

Eliminación: Caso Promedio: $O(k)$ | Peor Caso: $O(k)$.

Características Mecánicas: Cada nodo no guarda una palabra completa, sino un carácter. Los caminos desde la raíz hacia abajo forman las palabras del diccionario. El final de una palabra se marca explícitas con un booleano en el nodo final.

Ventajas y Limitaciones:

Mayor Ventaja: El tiempo de búsqueda no depende de la cantidad total de datos (n), sino únicamente del largo de la palabra (k).

Mayor Limitación: Alto consumo de memoria RAM debido a la enorme cantidad de punteros vacíos en caracteres poco compartidos.

Caso de Uso en la Industria: Algoritmos de Autocompletado de texto, correctores ortográficos de buscadores y tablas de enrutamiento IP de prefijo más largo.

1.5.Ficha Técnica 5: Heap (Montículo Min/Max)

Identificación: Árbol binario casi completo que cumple con la "propiedad de montículo".
En un Min-Heap el nodo padre es siempre menor o igual que sus hijos; en un Max-Heap es siempre mayor o igual.

Análisis de Complejidad (Big O):

Búsqueda (del elemento tope): Caso Promedio: $O(1)$ | Peor Caso: $O(1)$.

Inserción: Caso Promedio: $O(\log n)$ | Peor Caso: $O(\log n)$.

Eliminación (del elemento tope): Caso Promedio: $O(\log n)$ | Peor Caso: $O(\log n)$.

Características Mecánicas: Se implementa comúnmente usando un arreglo plano para ahorrar espacio en memoria. Las posiciones de los hijos se calculan matemáticamente ($2i + 1$ y $2i + 2$). Tras insertar o remover el tope, el árbol ejecuta un proceso de flotación ("heapify") para reordenar la propiedad rota.

Ventajas y Limitaciones:

Mayor Ventaja: Acceso instantáneo $O(1)$ al elemento máximo o mínimo del conjunto sin necesidad de ordenar la estructura entera.

Mayor Limitación: Buscar un elemento cualquiera que no sea la raíz requiere un recorrido completo, degradando la eficiencia a un costo de $O(n)$.

Caso de Uso en la Industria: Implementación interna de colas de prioridad en sistemas operativos (ej. planificación de procesos del CPU) y el algoritmo de ordenamiento Heapsort.